

CPT304 Workshop on CW

1 GitHub Workflow

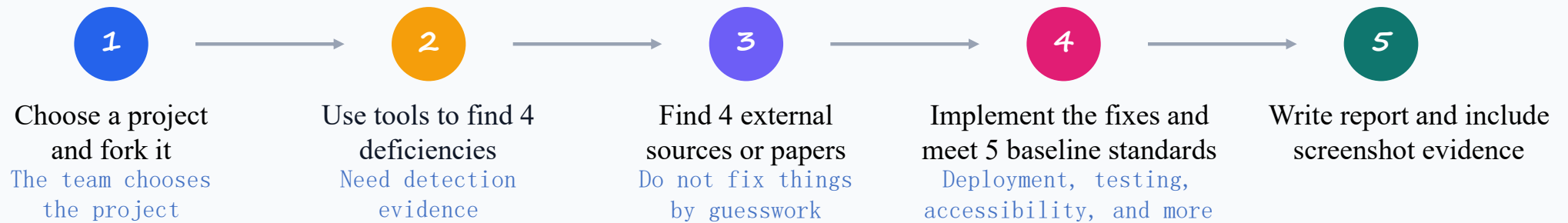
2 Baseline Standards

3 D-L-I Framework

ASSIGNMENT CONTEXT

What does this assignment actually require?

You need to turn a tiny open-source project into something that can be audited, researched, and deployed.

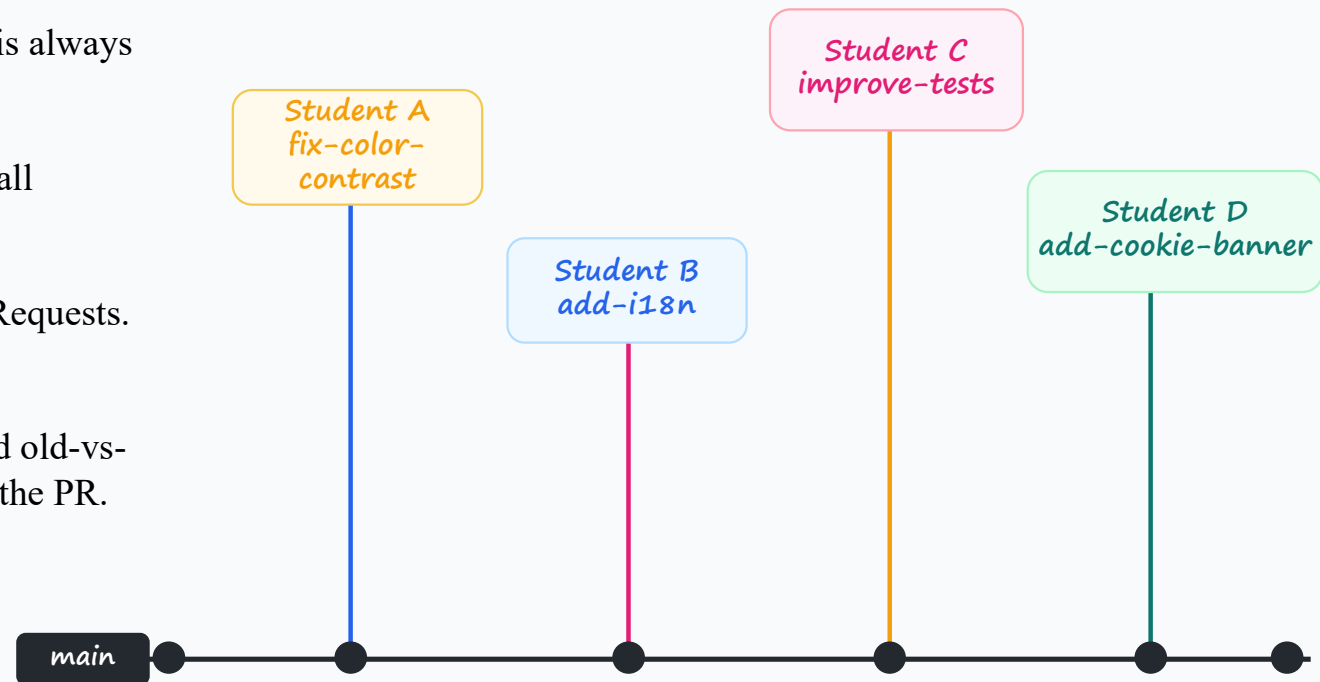


Today we focus on the 3 most important parts:
collaboration workflow, baseline tools, and report framework.

GitHub Workflow: Why you should not edit directly on main

For a four-person team, the safest way is to keep 'main' stable and put each task in a separate branch

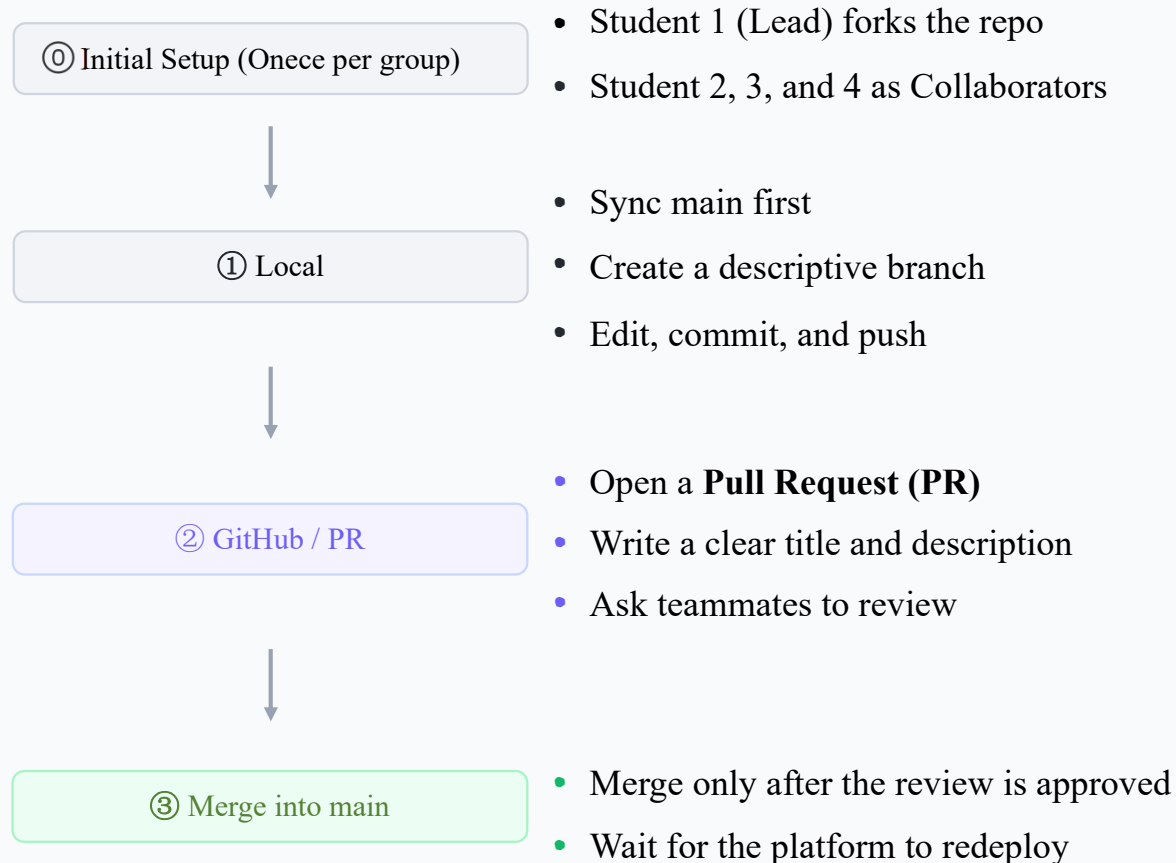
- main should only contain code that is always ready to deploy.
- One task = one branch. Do not mix all changes together.
- All merges should go through Pull Requests. Do not push directly to main.
- Review, discussion, screenshots, and old-vs-new code should all happen around the PR.



If all four people edit main directly, it is easy to get conflicts, rollbacks, and confusion about who broke the code

A standard flow: from local changes to merge

Think of local development, PR submission, and merge deployment as one chain, not three separate steps



```
1 git clone https://github.com/YourGroup/your-project.git
```

```
1 git checkout main
2 git pull origin main
3 git checkout -b fix-color-contrast
4 git add styles.css
5 git commit -m "docs: implement WCAG 4.5:1 ratio based on
web.dev research"
6 git push origin fix-color-contrast
```

A good PR description should answer at least four questions:

1. What did I change?
2. Why did I change it?
3. Where is the old-vs-new code?
4. What screenshots or preview links are included?

The 3 most common problems: conflicts, red checks, and rollback

Merge Conflict

1. Fetch the new changes.

```
1 git checkout main
2 git pull origin main
```

2. Go back to your feature branch and merge main into it:

```
1 git checkout fix-i18n-toggle
2 git merge main
```

3. Resolve the conflict: Git will mark the file. Open App.js, choose which code to keep, and remove the <<<< HEAD markers.

```
1 <<<<<< HEAD
2 buttonText = "提交"
3 =====
4 buttonText = t('submit')
5 >>>>>> main
```

4. Complete the merge.

```
1 git add App.js
2 git commit -m "chore: resolve merge conflict with main"
3 git push origin fix-i18n-toggle
```

Preview / Checks

1. **Vercel/Render:** When Student A opens a PR, a Preview Deployment is generated. Students B and C should click that link to test the fix in a browser before merging.

2. **Codecov:** If the PR reduces test coverage below 80%, the GitHub check will turn red. The team should reject the PR until Student A adds more tests.

Revert a Breaking Change

Scenario: You merged a PR that broke the live site. You need to fix it immediately to keep your "Green Days" status.

1. Find the "bad" commit hash in the log

```
1 git log --oneline
```

2. Revert it

```
1 git revert <commit-hash>
2 git push origin main
```

Five baseline standards: you must show evidence for every one

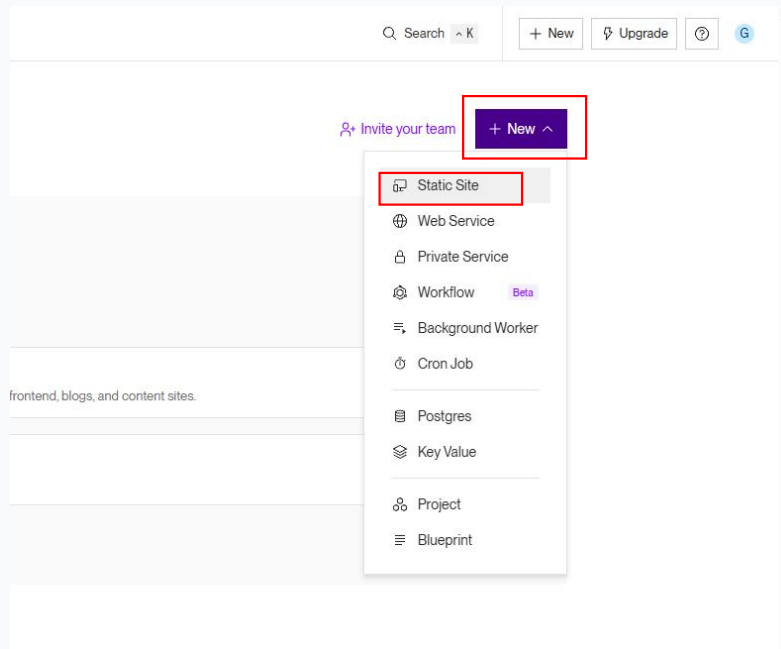
These five items are not optional bonus points. They are the minimum delivery baseline standards.



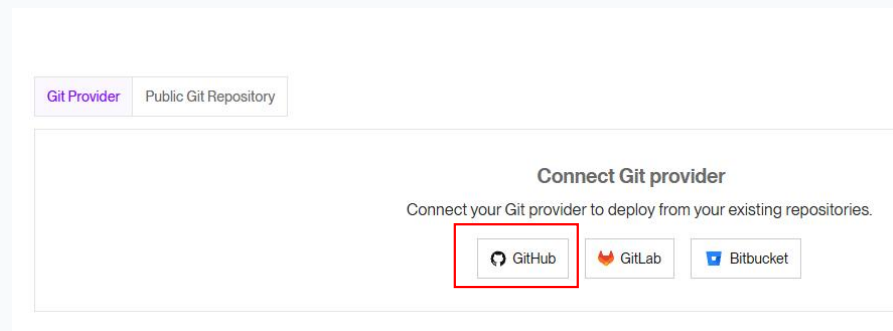
1) Live Uptime: Render and Vercel

- Connect the GitHub repository to Render or Vercel, here we use Render as an example

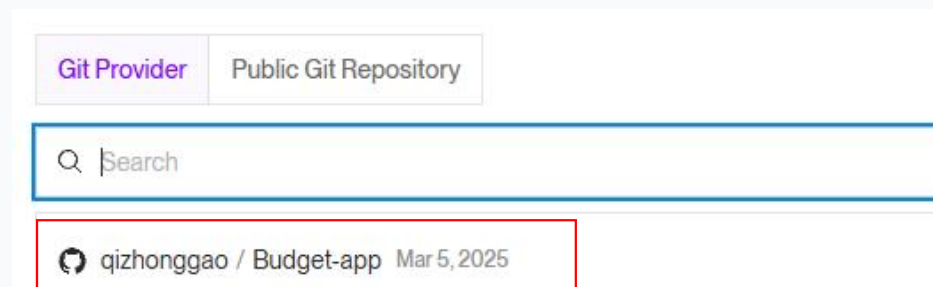
1. Open Render Dashboard and sign up an account.
2. In Homepage, click the 'New' button and choose 'Static Site'.



3. Click the Github option to connect your GitHub repository.



4. Then choose the repository you forked before.



1) Live Uptime: Render and Vercel

GitHub main

Auto Deploy

Public URL

Name
A unique name for your static site.

Branch
The Git branch to build and deploy.

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Build Command
Render runs this command to build your app before each deploy.

Publish Directory
The relative path of the directory containing built assets to publish. Examples: `./`, `./build`, `dist` and `frontend/build`.

Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

NAME_OF_VARIABLE

+ Add Environment Variable Add from .env

> **Advanced**

Deploy Static Site

5. In the setting, fill the 'Publish Directory' with a dot '.', then click button 'Deploy Static Site' below.

6. After the render processes the setup, you can view your page through the given link.

STATIC SITE

Budget-app

Service ID: `srv-d7b3929aae7s739ftsu0`

qizhonggao / Budget-app master

<https://budget-app-ydvf.onrender.com>

2) Test Coverage: local measurement, Codecov display, and checking

Think of it in two layers: local or CI first generates the coverage report, then Codecov shows the result in the PR and README.



Step 0: Setup Using GitHub Actions

Step 1: Output a Coverage Report

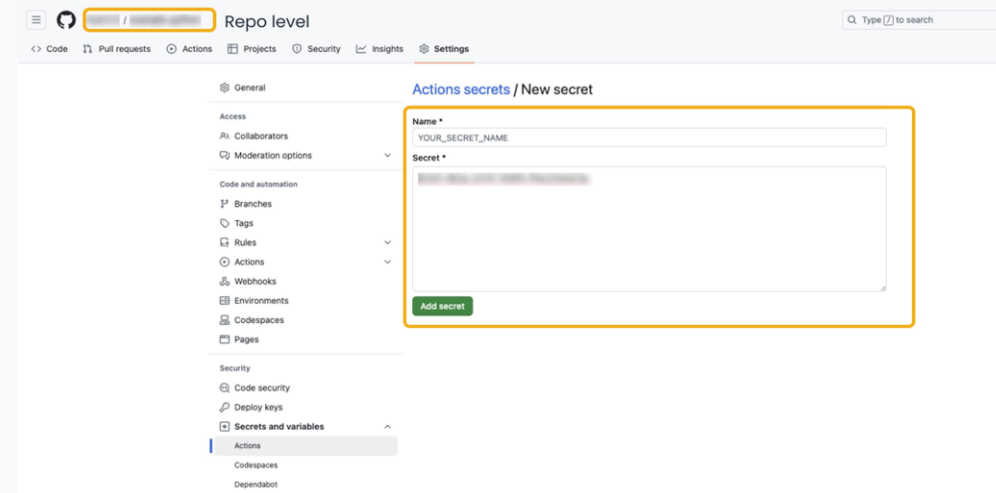
For example, using 'Jest':

```
1 npm install --save-dev jest
2 npx jest --coverage
```

Step 2: Add token as repository secret

Go to GitHub Project Repository -
> Settings -> Secrets and variables
-> Actions.

Add a secret name
'CODECOV_TOKEN'.



Step 3: Add Codecov to your GitHub Actions workflow yaml file

Sample actions workflow yaml file (based on different language to generate coverage reports) could be found in configure page.

2) Test Coverage: local measurement, Codecov display, and checking

Think of it in two layers: local or CI first generates the coverage report, then Codecov shows the result in the PR and README.

1/codecov/uploader

- renovate.json chore: add missing comma (#774)
- tsconfig.json fix: update types for xcode
- README.md

Codecov Uploader

circleci passing codecov 94% license scan passing lgtn alerts 0

codecov bot commented last week • edited

Codecov Report

Merging #150 (0f511b6) into main (fc9553f) will increase coverage by 0.01%.
The diff coverage is 100.00%.

	Coverage	Diff	
main	95.44		
#150	95.45	+0.01	
Files	709	709	
Lines	15201	15223	+22
Hits	14508	14530	+22
Misses	693	693	

Flag	Coverage Δ
unit	95.60% <100.00%> (+0.01%)
unit-latest-uploader	95.60% <100.00%> (+0.01%)

Simple Example of improving coverage

```
1 function shippingFee(isMember) {
2   if (isMember) return 0;
3   return 10;
4 }
5
6 // 只测一种情况:
7 expect(shippingFee(false)).toBe(10);
```

The test above only covers the non-member branch.

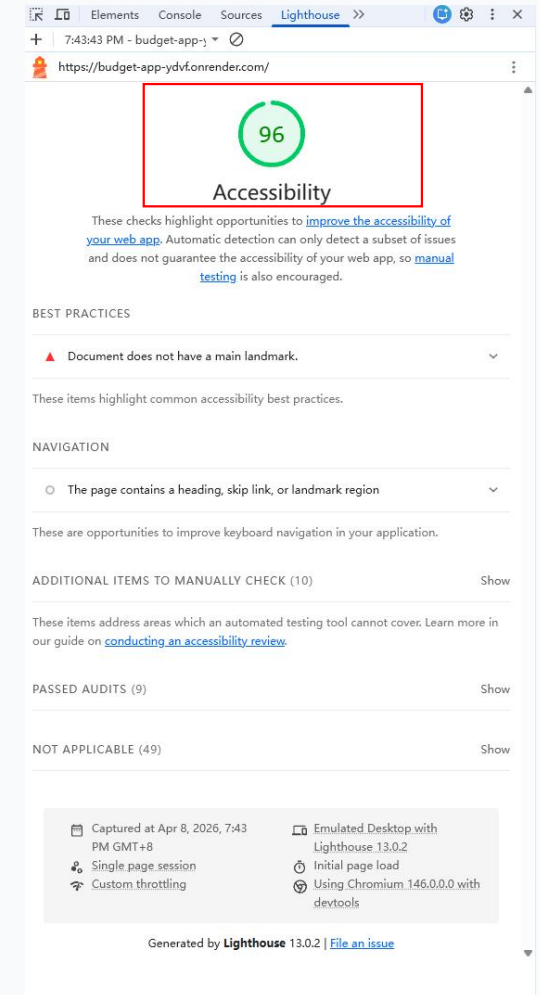
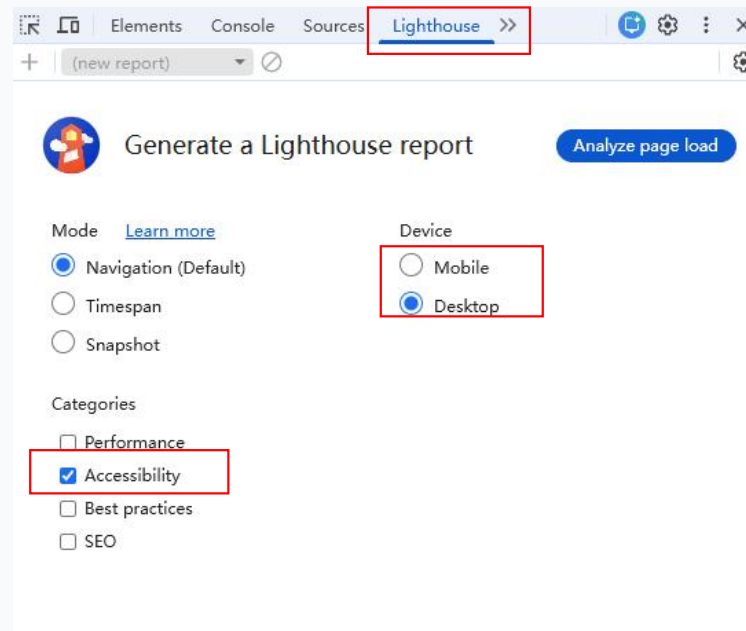
If you add one more member test case, the branch coverage becomes more complete.

3) Accessibility: Lighthouse 90+ is not just a number

It is really a set of accessibility audits. The score is only the starting point. What matters is whether you can explain why you fixed it this way.

- Lighthouse is Chrome's automated web audit tool.
- The Accessibility score comes from a weighted set of audits, and many items are pass/fail.
- Common reasons for losing points: low contrast, missing alt text, form inputs without labels, and messy heading order.

1. Click F12 on the keyboard.
2. Choose 'Lighthouse' in the menu bar.
3. Select 'Desktop' for the device.
4. Select 'Accessibility' only in the categories.
5. Then click 'Analyze page load' button. A score will be displayed.



4) Internationalization: externalized text + language switch

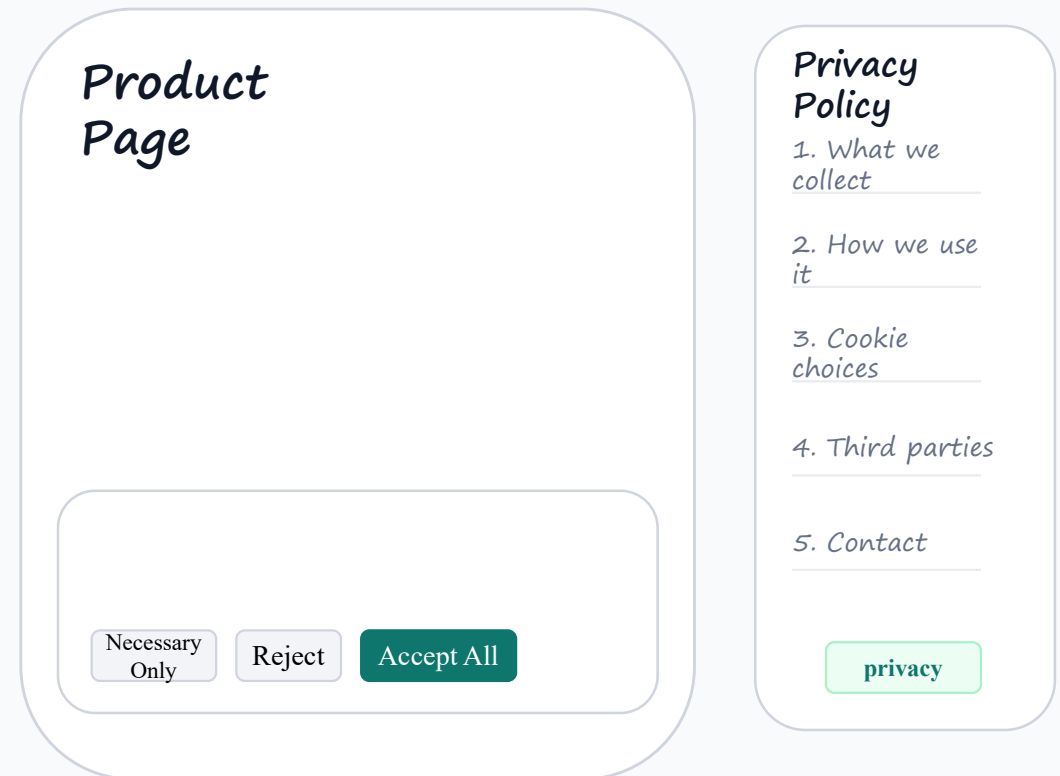
- Do not hard-code important text inside components.
- Change text into keys and read it with `t(...)`.
- At least two languages should cover key pages and buttons.

React examples

```
1 const { t, i18n } = useTranslation();
2
3 <button onClick={() =>
  i18n.changeLanguage('en')}>EN</button>
4 <button onClick={() =>
  i18n.changeLanguage('zh')}>中文</button>
5
6 <h1>{t('checkout.title')}</h1>
```

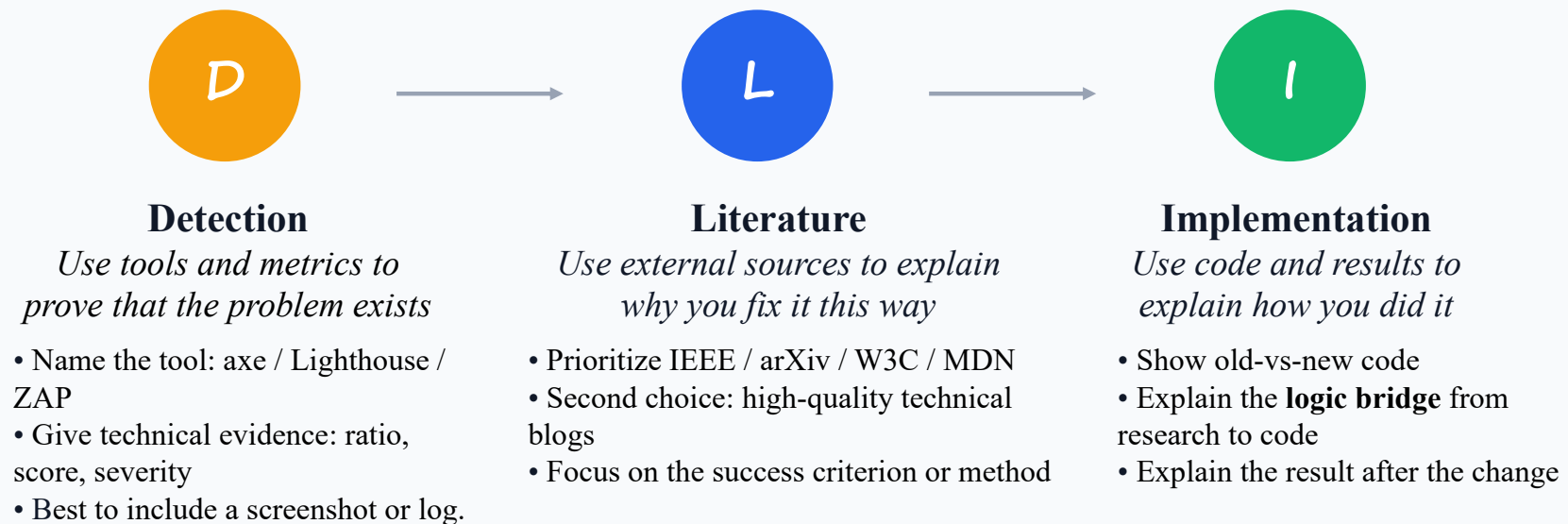
5) Legal Compliance: Cookie Banner + Privacy Policy page

- Cookie Banner: it appears on the first visit, the user can make a choice, and the system remembers the choice.
- Privacy Policy: it should be a separate page, not hidden in the README.
- The page should at least explain: what is collected, why it is collected, how it is used, and how to contact you.
- If there are non-essential scripts, a better approach is to enable them only after consent.



D-L-I Framework: What → Why → How

Each deficiency should follow the logic of What → Why → How.



Detection = What, Literature = Why, Implementation = How.

A complete example: button color contrast is too low

Detection

axe DevTools report: white text on a light-green background is only

1.45:1



=> Critical

Literature

WCAG 2.1 SC 1.4.3 says normal text must be at least 4.5:1

≥ 4.5:1

This gives the implementation a clear, testable target value.

Implementation

Change the background to #2D5A27. With white text, it reaches

7.21:1



=> Pass

Logic bridge: do not just make it darker randomly. Choose a color value that can be verified to pass the 4.5:1 threshold.

Old CSS

```
1 .submit-btn {  
2   background-color: #90EE90;  
3   color: #FFFFFF;  
4 } /* 1.45:1 */
```

New CSS

```
1 .submit-btn {  
2   background-color: #2D5A27;  
3   color: #FFFFFF;  
4 } /* 7.21:1 */
```

WRAP-UP

Finally, connect the three parts: what the real workflow looks like

